



**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential

C++ PROGRAMMING

(335)

State 2024

APPLICATION KNOWLEDGE:

StarCargo Express Labels _____ (400 points)

TOTAL POINTS _____ (*400 points*)

Test Time: 90 minutes

Your company, StarCargo Express, has asked you to create a program that will create sheets of mailing labels for all its customers.

Requirements:

1. You must create a C++ console application named `CPP_335_ContestantNumber`, where `ContestantNumber` is your BPA assigned contestant number (including dashes). For example, `CPP_335_01_2345_6789`.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. When application starts, application shall read all the records stored in provided common separated value text file, “Customers.txt”.
 - a. The program should validate the input file to make sure that it exists. If it is not found, print an error message, and terminate the program.
 - b. If the input file is valid the program should create a data structure of Customer objects.
4. StarCargo Express requires all programs to be modular and reusable, so you are required to create a class called “Customer”. “Customer” class requirements:
 - a. Six (6) member variables to account for each of the six (6) fields below:
 - i. Last Name
 - ii. First Name
 - iii. Street Address
 - iv. City
 - v. State
 - vi. Zip Code
 - b. One overloaded constructor that will take in the comma separated record for each customer.
 - c. Setter and getter methods for all 6 fields (see above).
 - d. A “Label” method that returns a single string that contains the record in mailing label format.
 - i. The “last name, first name” on the first line
 - ii. The “street address” on the second line
 - iii. The “city, state zip” on the third line (See Figure 1 for an example).
5. The program will print all the mailing labels in one column to the screen.
 - a. The labels must be formatted like Figure 1 with two blank lines between each record.
 - b. The labels must be grouped by zip code and then sorted by last name.
 - i. For example, all addresses with the zip code “12345” would appear together and before “23456”.
 - ii. And within zip code “12345”, “Jones” would appear before “Smith”.
6. The code must be commented, and the variables must have proper and consistent naming conventions.

Johnson, Emily
123 Maple Street
Columbus OH 43201

← Figure 1: Sample Mailing Label: the first name and Last Name on first line separated by a comma and a space. The street address will be on the second line of the label. The city, state, and zip will be on the last line of the label separated by spaces.

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 pts

The project is named according to the naming conventions _____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB _____ 10 pts

If the program does not execute, then the remaining items below receive a score of zero.

Application loads data from “Customers.txt” and displays error message if the file is not loaded _____ 30 pts

Labels are printed to the screen in the proper format _____ 30 pts

Labels are grouped by zip code and sorted by last name within each zip code _____ 50 pts

Source Code Review

Code is commented at the top, for each function, and as needed _____ 20 pts

Code uses reasonable and consistent variable naming conventions _____ 20 pts

Customer class exists with all required components _____ 70 pts

Customer objects are instantiated by passing comma separated string to the constructor _____ 50 pts

The label method returns a single properly formatted string _____ 50 pts

Main function prints the labels to the screen using logic to group by zip code _____ 50 pts

Total Points: _____ 400 pts

NOTE: The “Customers.txt” file providers to the graders is different from the one provided to the students for testing purposes. Replace the file with the one provided to the students on their submissions to ensure proper functionality with a similarly formatted, but different data set.

Sample Solution: Competitor code may vary.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <sstream>

// BPA Contestant Number: CPP_335_01_2345_6789

class Customer {
private:
    std::string lastName, firstName, streetAddress, city, state, zipCode;

public:
    Customer(std::string record) {
        std::istringstream ss(record);
        getline(ss, lastName, ',');
        getline(ss, firstName, ',');
        getline(ss, streetAddress, ',');
        getline(ss, city, ',');
        getline(ss, state, ',');
        getline(ss, zipCode);
    }

    // Getters
    std::string getLastName() const { return lastName; }
    std::string getFirstName() const { return firstName; }
    std::string getStreetAddress() const { return streetAddress; }
    std::string getCity() const { return city; }
    std::string getState() const { return state; }
    std::string getZipCode() const { return zipCode; }

    // Setters
    void setLastName(const std::string& ln) { lastName = ln; }
    void setFirstName(const std::string& fn) { firstName = fn; }
    void setStreetAddress(const std::string& sa) { streetAddress = sa; }
    void setCity(const std::string& c) { city = c; }
    void setState(const std::string& s) { state = s; }
    void setZipCode(const std::string& zc) { zipCode = zc; }

    std::string Label() const {
        return lastName + ", " + firstName + "\n" + streetAddress + "\n" + city + ", " + state + " " + zipCode;
    }

    // For sorting purpose
    friend bool operator<(const Customer &a, const Customer &b) {
        if (a.zipCode == b.zipCode) {
            return a.lastName < b.lastName;
        }
        return a.zipCode < b.zipCode;
    }
};
```

```
int main() {
    std::ifstream file("Customers.txt");
    if (!file) {
        std::cerr << "Error: File not found." << std::endl;
        return 1;
    }

    std::vector<Customer> customers;
    std::string line;
    while (getline(file, line)) {
        customers.emplace_back(line);
    }

    std::sort(customers.begin(), customers.end());

    for (const auto& customer : customers) {
        std::cout << customer.Label() << "\n\n";
    }

    return 0;
}
```